This is the Title

Author One Author Two Author Three
Department of Departments, University Name, City, Country
{author1, author2, author3}@university.edu

CONTENTS

I Introduction 1

II Background & Related Works 1
II-A Centralized . . . 2
II-B Decentralized . . . 2

III Agentic Communication 2
III-A Mesh Topology . . . 2
III-B Message Schema . . . 2
III-C Grounding Intent Decomposition . . . 3
III-D Knowledge-Graph Coordination . . . 3
III-E Fault Detection and Dynamic Rerouting 3
III-F Connecting to the Methodology . . . 3

IV Methodology 3
IV-A Task Decomposition Proof . . . 3
IV-A1 Algorithmic Description . . . 4
IV-B Mesh Traversal Proof . . . 4
IV-B1 Algorithmic Description . . . 4
IV-C Metrics . . . 4
IV-D Experimental Setup . . . 5

V Results 5

VI Conclusion 5

***Abstract*—Abstract**
***Index Terms*—Keywords**

I. INTRODUCTION

In this paper, we propose a decentralized mesh architecture for agentic communication that decomposes natural language intents into modular tasks, distributes them across a network of autonomous agents, and coordinates their execution through a shared knowledge graph. The mesh enables each agent to select appropriate tools, invoke large language model services, and exchange results without a central controller, thereby improving scalability, fault tolerance, and privacy.

Our work also introduces a novel grounding intent decomposition technique for hierarchical task delegations. The system translates high-level user requests into concrete actions, monitors progress, and dynamically reroutes work when nodes fail or become overloaded. Experimental evaluation on benchmark workflows demonstrates that the mesh reduces latency and increases success rates compared with monolithic orchestrators, illustrating the benefits of a distributed, self-composing approach to intelligent automation.

Problem. Centralized orchestrators create bottlenecks, single points of failure, and privacy risks as agentic workloads grow in complexity. A single controller cannot scale to meet the demands of large, dynamic multi-agent pipelines.

Limitations of existing solutions. Existing frameworks route all task coordination through a central node, meaning controller failure halts the entire pipeline. Privacy is further compromised as all agent context passes through one point. Prior decentralized attempts lack formal guarantees on task completion and mesh traversal.

Our contributions are as follows:

- A **decentralized mesh architecture** for agentic communication that relies on a shared knowledge graph and eliminates any central controller.
- A **high-level orchestration algorithm** (Algorithm **??**) that ties together intent decomposition, mesh routing, and result aggregation.
- A **task-decomposition algorithm** (Algorithm IV-A1) that produces a finite, grounded sub-task DAG from a natural-language intent.
- A **mesh-routing algorithm** (Algorithm IV-B1) that learns edge weights, enforces reliability thresholds, and supports proactive load-based redirects.
- A **dynamic fault-rerouting mechanism** that detects missed heartbeats and re-queues in-flight subtasks within a bounded recovery time.
- Formal **proofs of decomposition completeness, mesh liveness, and execution safety** (Sections IV-A and IV-B).
- A comprehensive **evaluation framework** with quantitative metrics (latency, success rate, fault-recovery time, knowledge-graph consistency, message overhead, delegation depth, edge-weight convergence) and empirical results showing reduced end-to-end latency and higher task success rates compared with monolithic baselines.

The rest of this paper is organized as follows. Section II reviews background and related work. Section III describes agentic communication in the mesh. Section IV presents our methodology, formal proofs, and experimental setup. Section V reports results. Section VI concludes and outlines future work.

II. BACKGROUND & RELATED WORKS

Multi-agent orchestration systems fall broadly into two paradigms: centralized, where a single controller mediates all task routing and agent coordination, and decentralized, where agents communicate and self-organize without a global author-

ity. We review both, identifying the strengths and limitations relevant to our work.

A. Centralized

Centralized orchestration frameworks designate a single planner or manager node that dispatches tasks to worker agents and aggregates results.

LangChain and LangGraph construct directed computation graphs in which a central planner sequences or parallelizes tool calls. While easy to reason about and debug, the planner represents a single point of failure and cannot support peer-to-peer agent negotiation. We discuss LangChain/Graph because they provide a clear baseline for orchestrating LLM tool usage; our work builds on the idea of graph-based planning but replaces the monolithic planner with a decentralized mesh that eliminates the single point of failure and enables dynamic routing.

AutoGen adopts a conversation-driven model in which a central manager routes messages between agents. The manager's central role creates a bottleneck under high concurrency and prevents agents from self-organizing in its absence. We reference AutoGen to illustrate the limitations of a single manager for scaling multi-agent dialogue; our mesh architecture removes this bottleneck by allowing agents to negotiate and forward tasks autonomously.

CrewAI organizes agents into role-based crews governed by a sequential or hierarchical process object. Controller failure terminates the entire crew, and there is no mechanism for dynamic rerouting of in-flight tasks. CrewAI is cited because it demonstrates structured role composition, which we extend by allowing roles to be discovered and re-routed at runtime without a fixed hierarchy.

OpenAI Assistants API mediates all tool use and memory through a managed central server, introducing vendor lock-in and precluding mesh-style routing. In summary, centralized systems trade scalability and resilience for simplicity of control flow. We mention this API to highlight the contrast between proprietary, server-side orchestration and our open, self-healing mesh that operates without external lock-in.

B. Decentralized

Decentralized approaches distribute coordination responsibility across agent nodes, removing the dependency on a single controller.

JADE is a FIPA-compliant agent platform supporting distributed containers and peer message passing. While mature and standards-based, JADE is not LLM-native and provides no natural language intent decomposition or knowledge graph coordination. We include JADE as a classic example of decentralized agent communication; our contribution adapts the decentralized paradigm to LLM-driven intent decomposition and shared knowledge graphs.

Swarm (OpenAI) introduces a lightweight handoff protocol enabling direct agent-to-agent task transfer. However, it provides no shared knowledge graph, no formal routing guarantees, and no fault recovery mechanism. Swarm is discussed because it demonstrates the feasibility of peer-to-peer handoff; we extend this idea with a learned routing mesh, explicit reliability scores, and a shared blackboard for consistent state.

AgentVerse and MetaGPT support role-playing multi-agent workflows with limited peer communication, but still rely on a meta-agent or fixed role topology rather than a true dynamic mesh. These systems are cited to show the evolution from static role assignment toward our approach, which allows agents to discover and assume roles dynamically based on mesh routing.

Gossip and epidemic protocols from the distributed systems literature inspire our self-healing mesh routing design but were not designed for LLM-based task execution or semantic intent decomposition. We reference these protocols to justify the use of probabilistic, fault-tolerant propagation mechanisms, and then adapt them to the semantic domain of LLM intents.

Knowledge graph-based coordination approaches such as KGAgent use structured graphs for shared agent memory but lack dynamic task decomposition, grounding, and rerouting. Our work addresses all of these gaps simultaneously. KGAgent is mentioned to highlight prior attempts at shared semantic state; we extend this by coupling the knowledge graph with on-the-fly decomposition and a mesh that can reroute tasks when agents fail.

III. Agentic Communication

We define **agentic communication** as structured, semantic message passing between autonomous LLM-backed agents that exchange task context, tool results, and routing metadata without a central broker. This section refines the description introduced in the background (Section II) and connects it to the formal algorithms and guarantees presented in the methodology (Section IV).

A. Mesh Topology

Agents are modeled as vertices of a weighted directed graph $G = (V, E, W)$. Each edge $e \in E$ carries a weight $W(e) \in \mathbb{R}^+$ that reflects latency, load, and a reliability score derived from the routing algorithm (Algorithm IV-B1). Unlike the static topologies of systems such as JADE or Swarm (see Section II-B), our mesh continuously adapts edge weights through reinforcement (reward/penalty) and pruning, guaranteeing the liveness and safety properties proved in Section IV-B.

B. Message Schema

Every inter-agent message conforms to a minimal schema consisting of:

- **Header**: a structured block containing (i) the unique identifier of the sending agent, (ii) the unique identifier of the intended receiving agent, and (iii) a message type flag that specifies the purpose of the transmission – `Task` (request to execute a sub-task), `Result` (return of execution output), `Redirect` (instruction to forward the sub-task to another agent), or `Heartbeat` (liveness probe).

- **Payload**: the sub-task identifier, optional tool input, and a **knowledge-graph delta** that records any new triples or property-graph updates produced by the sender.
- **Routing Metadata**: the current edge weight, reliability estimate, and a routing snapshot (as defined in Algorithm IV-B1).

The delta is merged into the shared knowledge graph $\mathcal{KG}$ using vector-clock-based conflict resolution, ensuring eventual consistency without a central lock – a direct improvement over the centralized stores used by LangChain/Graph and the OpenAI Assistants API.

C. Grounding Intent Decomposition

An incoming high-level intent I is first processed by the **decomposition stage** of the high-level orchestration algorithm (Algorithm **??**). The decomposition routine (Algorithm IV-A1) produces a finite set S of sub-intent specifications and builds a directed acyclic graph $\mathcal{T} = (V_T, E_T)$. Each node $v \in V_T$ stores:

- a concrete tool binding b_v,
- an expected output schema σ_v,
- a Boolean success predicate p_v.

Grounding guarantees that every sub-task is verifiable before it enters the mesh, addressing the hallucination risk observed in chain-of-thought prompting and the lack of formal grounding in systems such as AgentVerse and MetaGPT.

D. Knowledge-Graph Coordination

The shared knowledge graph $\mathcal{KG}$ acts as a distributed blackboard. Agents read from $\mathcal{KG}$ to obtain context for subsequent subtasks and write results as property-graph nodes or RDF triples. Consistency is eventual: reads may be stale by at most δ seconds, a bound measured by the **Knowledge-Graph Consistency** metric (Section IV-C). Vector clocks attached to each sub-task node enable deterministic conflict resolution, eliminating the need for a central write lock that hampers scalability in centralized frameworks.

E. Fault Detection and Dynamic Rerouting

Heartbeat messages are broadcast every τ seconds. If a node misses k consecutive heartbeats it is marked FAILED. The mesh then invokes the routing algorithm (Algorithm IV-B1) to re-queue in-flight subtasks to a healthy neighbour. This mechanism directly addresses the single-point-of-failure limitation of centralized orchestrators (LangChain, AutoGen, CrewAI) and the lack of proactive load-based redirection in Swarm.

When an agent's local queue length exceeds a threshold θ, it emits a REDIRECT message. The receiving agent evaluates the same routing function used for failure recovery, ensuring that delegation respects the same reliability and edge-weight criteria. The **Delegation Depth** and **Edge-Weight Convergence** metrics (Section IV-C) quantify the effectiveness of this dynamic rerouting.

F. Connecting to the Methodology

The three algorithms introduced in Section IV—the high-level orchestration (Algorithm **??**), the task-decomposition routine (Algorithm IV-A1), and the mesh-routing procedure (Algorithm IV-B1)—operate together to realise the agentic communication model described above. The formal proofs of **Decomposition Completeness** (Section IV-A) and **Mesh Liveness / Execution Safety** (Section IV-B) guarantee that every grounded sub-task will eventually reach a capable agent and will be executed exactly once, even in the presence of failures or load-induced redirects.

By explicitly linking the communication primitives to the algorithms and proofs, this section provides a cohesive narrative that bridges the related-work analysis (Section II) with the technical contributions of the paper.

IV. METHODOLOGY

This section provides formal guarantees for the two core algorithmic components of the mesh (Sections refsec:task-decomposition-proof and refsec:mesh-traversal-proof), defines the evaluation metrics (Section refsec:metrics), and describes the experimental setup used to validate our claims empirically (Section refsec:experimental-setup).

A. Task Decomposition Proof

Lemma 1 (Grounding Soundness)**.** *For every subtask $t_i \in \mathcal{T}$, there exists a tool binding b_i such that $b_i(input_i)$ produces an output matching the expected schema σ_i.*

Theorem 2 (Decomposition Completeness)**.** *Given a well-formed intent I that belongs to the grammar $mathcalG_I$, and assuming the decomposition routine returns a finite set S of sub-intent specifications, the algorithm constructs a directed acyclic graph $mathcalT = (V_T, E_T)$ such that a topological execution of $mathcalT$ produces a result that satisfies the success predicate of I.*

Proof. Given $I \in \mathcal{G}_I$, the decomposition step enumerates all sub-intents $S = \{s_1, \ldots, s_n\}$. Assume (A1) S is finite – this follows from the finiteness of $\mathcal{G}_I$. Assume (A2) each s_i contains an explicit dependency set $textdeps(s_i) \subseteq \{s_1, \ldots, s_n\}$. The construction adds a node v_i for each s_i and creates edges $(v_j \to v_i)$ for every $v_j \in textdeps(s_i)$. Because dependencies are only on previously defined sub-intents, no cycle can be introduced; thus $mathcalT$ is acyclic. Topological sorting yields an order $sigma$ where all dependencies of a node appear earlier in $sigma$. Executing nodes in $sigma$ guarantees that when a node is processed, all its inputs are already available, satisfying the precondition of each tool binding. Consequently the final blackboard contains values for all v_i, and the composition of these values fulfills the success predicate of I. □

Algorithm 1 Task Decomposition and Execution

Require: High-level intent I, maximum delegation depth D_{max}

$G \leftarrow$ empty directed acyclic graph
$B \leftarrow$ empty blackboard
function DECOMPOSE(I) ▷ LLM or rule-based
    return list of sub-intent specifications $S = \{s_1, \ldots, s_n\}$
end function
$S \leftarrow$ DECOMPOSE(I)
for each $s \in S$ **do**
    create node v with fields intent, domain, deps from s
    add v to G
end for
$Ready \leftarrow$ nodes in G with no unsatisfied dependencies
while $Ready \neq \emptyset$ **do**
    select v from $Ready$ (deterministic order)
    if local tool for v exists **then** ▷ tool binding b
        execute b, store result in $B[v.id]$
        mark v as DONE
    else ▷ delegate
        if $depth(v) < D_{max}$ **then**
            execute tool, store result in $\mathcal{B}[v.id]$
        else ▷ route via mesh
            $res \leftarrow$ MESHROUTE(v, $\mathcal{M}$) ▷ Algorithm IV-B1
            store res in $\mathcal{B}[v.id]$
        end if
    end if
end while
$\mathcal{R} \leftarrow$ aggregate results from $\mathcal{B}$
return $\mathcal{R}$

1) Algorithmic Description:

B. Mesh Traversal Proof

Theorem 3 (Mesh Liveness)**.** *In a mesh $G = (V, E)$ with at least two agents, any subtask t_i is routed to a healthy agent that advertises the required tool within $mathcalO($ $mathrmdiam(G))$ hops, provided such an agent exists.*

Theorem 4 (Execution Safety)**.** *No subtask t_i is executed more than once. The mesh records a routing snapshot per node and the task engine marks a node "DONE" only after a successful local or delegated execution, preventing duplicate processing even under concurrent routing.*

Proof. Assume (B1) the mesh graph G is connected and contains at least two agents. Assume (B2) each agent maintains a non-negative edge weight $w(e)$ and a reliability score $r(a) \in [0, 1]$. Assume (B3) the routing function RESOLVEAGENT (Algorithm IV-B1) selects the candidate a^* that maximises the product $w(a_{cur}, a) \cdot r(a)$ among agents not yet visited for the current node. For any subtask t_i requiring tool t, let A_t be the set of agents advertising t. Because G is connected, there exists a path from the current agent a_{cur} to some $a \in A_t$. By (B3) the algorithm will eventually select such an a unless all candidates have been visited, which cannot happen before reaching a healthy agent because the visited set is cleared for each new node. Each successful routing updates the edge weight with a positive reward (see Algorithm IV-B1), ensuring that the weight of the chosen edge does not decrease below a positive constant. Consequently the number of hops required to reach a healthy agent is bounded by the diameter of G, yielding the $mathcalO($ $mathrmdiam(G))$ bound. For safety, assume (B4) a node is marked "DONE" only after the execution step returns success. The mesh records a routing snapshot for each node; any subsequent attempt to execute the same node will find the "DONE" status and abort. Since the snapshot is immutable after success, duplicate executions cannot occur, establishing the uniqueness claim. □

Algorithm 2 Mesh Routing for a Sub-Task

Require: Node identifier n, required tool t, current agent key a_{cur}

$Visited \leftarrow \emptyset$
function RESOLVEAGENT($domain, t, n$)
    $Candidates \leftarrow$ agents advertising t in $domain$
    $Candidates \leftarrow Candidates \setminus Visited$
    if $Candidates = \emptyset$ **thenreturn** $\perp$
    end if
    compute score $s(a) = w(a_{cur}, a) \cdot r(a)$ for each $a \in Candidates$
    $a^* \leftarrow \arg\max_a s(a)$
    $Visited \leftarrow Visited \cup \{a^*\}$
    return a^*
end function
$a \leftarrow$ RESOLVEAGENT($domain, t, n$)
if $a = \perp$ **then** ▷ no healthy agent
    mark node n as FAILED
    return
end if
execute tool t on agent a
if execution succeeds **then** ▷ update edge weight
    $w(a_{cur}, a) \leftarrow (w(a_{cur}, a) \cdot \alpha) + \beta$ ▷ reward
    $a_{cur} \leftarrow a$
    mark node n as DONE
else ▷ penalty and possible retry
    $w(a_{cur}, a) \leftarrow (w(a_{cur}, a) \cdot \alpha) - \beta$
    if failure count $<$ threshold **then**
        retry with new a from RESOLVEAGENT(...)
    end if
end if

1) Algorithmic Description:

C. Metrics

The evaluation framework records a comprehensive set of quantitative indicators. Each metric is derived directly

from the run-time artefacts (timestamps, blackboard entries, routing snapshots and edge-weight logs) without referring to implementation details.

- **End-to-End Latency** (L): measured as the difference between the timestamp recorded when an intent is first submitted and the timestamp when the final result is written to the blackboard. The value is expressed in milliseconds; lower values indicate a faster overall response.
- **Task Success Rate** (TSR): the proportion of benchmark intents for which every sub-task reaches the "DONE" state. Formally, $TSR = \frac{\text{number of intents with all nodes DONE}}{\text{total number of intents}}$. Higher values reflect more reliable orchestration.
- **Fault Recovery Time** (FRT): the elapsed time between the detection of a node failure (recorded in the routing snapshot) and the moment a replacement agent successfully completes the affected sub-task. Shorter recovery times demonstrate robustness to agent loss.
- **Throughput** (TP): computed as the number of intents processed per second when the system is subjected to concurrent load. It is obtained by dividing the total number of intents completed in a measurement window by the window duration.
- **Knowledge-Graph Consistency** (KGC): the fraction of reads from the shared blackboard that return a value whose age is within a predefined staleness bound δ. Consistency is calculated as $KGC = \frac{\text{reads within } \delta}{\text{total reads}}$.
- **Message Overhead** (MO): the total count of inter-agent messages (routing requests, delegations and acknowledgements) divided by the number of sub-tasks for a given intent. This normalises communication cost across workloads of different sizes.
- **Delegation Depth** (DD): the deepest recursion level observed when a sub-task is handed off to a neighbour. The metric is bounded by the mesh configuration that limits the maximum delegation depth; lower depths indicate more direct routing.
- **Edge-Weight Convergence** (EWC): the statistical variance of the learned edge-weight values after a fixed number of runs. A low variance signals that the routing policy has stabilised, whereas a high variance suggests ongoing exploration.
- **Composite Score** (S): a weighted aggregation of the above indicators, using the default weight vector {S:0.2, D:0.2, W:0.2, R:0.2, O:0.2}. Each component (latency, success, weight stability, reliability and overhead) is normalised to $[0, 1]$ before applying the weights, yielding a single scalar that facilitates ranking of different system configurations.

D. Experimental Setup

V. Results

VI. Conclusion